

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-213Б-23

Студент: Германенко М. И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 26.12.24

Постановка задачи

Исследовать два аллокатора памяти: необходимо реализовать два алгоритма аллокации памяти и сравнить их по следующим характеристикам:

- Фактор использования
- Скорость выделения блоков
- Скорость освобождения блоков
- Простота использования аллокатора

Требуется создать две динамические библиотеки, реализующие два аллокатора, соответственно. Библиотеки загружаются в память с помощью интерфейса ОС (dlopen / LoadLibrary) для работы с динамическими библиотеками. Выбор библиотеки, реализующей аллокатор, осуществляется чтением первого аргумента при запуске программы (argv[1]). Этот аргумент должен содержать путь до динамической библиотеки (относительный или абсолютный).

Если аргумент не передан или по переданному пути библиотеки не оказалось, то указатели на функции, реализующие API аллокатора ниже, должны быть присвоены функциям, которые оборачивают системный аллокатор ОС (mmap / VirtualAlloc) в этот API. Эти аварийные оберточные функции должны быть реализованы внутри программы, которая загружает динамические библиотеки (см. пример на GitHub Gist). Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям malloc и free (realloc, опционально).

Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными стандартными средствами ядра (mmap / VirtualAlloc). Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше.

Каждый аллокатор должен обладать следующим интерфейсом (могут быть отличия в зависимости от особенностей алгоритма):

- Allocator* allocator_create(void *const memory, const size_t size) (инициализация аллокатора на памяти memory размера size);
- void allocator_destroy(Allocator *const allocator) (деинициализация структуры аллокатора);
- void* allocator_alloc(Allocator *const allocator, const size_t size) (выделение памяти аллокатором памяти размера size);
- void allocator_free(Allocator *const allocator, void *const memory) (возвращает выделенную память аллокатору);

Вариант 8. Списки свободных блоков (наиболее подходящее) и алгоритм Мак-Кьюзика-Кэрелса;

Общий метод и алгоритм решения

Использованные системные вызовы:

- dlopen(const char *filename, int flag) — загружает разделяемую библиотеку в адресное пространство процесса.
- dlsym(void *handle, const char *symbol) — получает адрес символа (функции или переменной) из разделяемой библиотеки.
- dlclose(void *handle) — выгружает разделяемую библиотеку.
- mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset) — отображает блок памяти в адресное пространство процесса.
- munmap(void *addr, size_t length) — удаляет отображение памяти.
- write(int fd, const void *buf, size_t count) — записывает данные в файловый дескриптор.

- `read(int fd, void *buf, size_t count)` — читает данные из файлового дескриптора.

Описание метода решения:

Программа реализует выделение, освобождение и управление памятью с использованием пользовательского аллокатора, который динамически подключается через разделяемую библиотеку. Для демонстрации работы аллокатора используются операции выделения и освобождения памяти, а также преобразование указателей в строку.

Основной алгоритм:

Инициализация программы:

1. Использовать `dlopen` для загрузки разделяемой библиотеки, содержащей функции аллокатора.
2. Проверить успешность загрузки библиотеки. Если библиотека недоступна, использовать системные вызовы `mmap` и `munmap` для работы с памятью (fallback-аллокатор).
3. С помощью `dlsym` получить адреса следующих функций из библиотеки:
 - `allocator_create` — создание аллокатора.
 - `allocator_destroy` — уничтожение аллокатора.
 - `allocator_alloc` — выделение памяти.
 - `allocator_free` — освобождение памяти.

Работа с памятью:

4. Вызвать `allocator_create` для создания нового объекта аллокатора, передав ему заранее выделенный блок памяти.
5. Если аллокатор успешно создан, начать работу:
 - Выделить блок памяти размером 64 байта с помощью `allocator_alloc`.
 - Вывести адрес выделенного блока в консоль в формате `0x` с помощью функции преобразования указателей в строку.
 - Освободить блок памяти с помощью `allocator_free`.
6. Повторить те же действия для блока памяти размером 128 байт.

Завершение работы:

7. Вызвать `allocator_destroy` для освобождения всех ресурсов аллокатора.
8. Если использовался fallback-аллокатор, освободить всю выделенную память через `munmap`.
9. Закрывать разделяемую библиотеку с помощью `dlclose`.

Ключевые аспекты использования памяти:

1. Безопасность: Все указатели проверяются на корректность перед использованием, исключая возможные ошибки сегментации.
2. Управление ресурсами: При завершении программы все выделенные ресурсы (блоки памяти, разделяемая библиотека) освобождаются, исключая утечки памяти.
3. Модульность: Разделение функционала аллокатора в отдельной библиотеке позволяет легко обновлять или заменять его реализацию без изменения основного кода программы.

Итог: Программа демонстрирует эффективное и безопасное управление памятью с использованием модульного подхода и динамической загрузки пользовательского аллокатора.

Описание алгоритмов и их сравнение:

В рамках данной работы были реализованы и исследованы два алгоритма аллокации памяти: аллокатор на основе свободного списка (FreeList Allocator) и аллокатор МакКьюзика (McKusick Allocator). Оба аллокатора представляют собой различные подходы к управлению памятью, которые отличаются стратегией выделения и освобождения блоков, а также своей эффективностью и применимостью в реальных задачах. Аллокатор FreeList использует связанный список свободных блоков для хранения информации о доступной памяти, что позволяет быстро находить подходящий блок, минимизируя фрагментацию памяти. Аллокатор МакКьюзика, в свою очередь, делит память на фиксированные классы размера и управляет ею на основе бин-подобной структуры, что делает его эффективным для приложений, работающих с предсказуемыми размерами данных.

Для оценки работы аллокаторов был проведен процесс тестирования, включающий выполнение операций выделения и освобождения блоков различного размера с измерением времени выполнения каждой операции. Для повышения точности эксперимента использовались многократные повторения тестов, а также замеры для случайных и последовательных размеров блоков. Дополнительно проверялась корректность освобождения памяти и способность аллокаторов возвращать память в повторное использование. Тестирование выполнялось на реальном времени работы программ с использованием системного вызова `clock_gettime` для измерения временных характеристик.

Основным подходом тестирования стало моделирование сценариев, близких к реальным приложениям, включая частое выделение и освобождение блоков с типичными размерами. Это позволило оценить как общую производительность аллокаторов, так и их стабильность при длительном использовании. Дополнительно анализировался фактор использования памяти, определяющий степень фрагментации в процессе работы.

Результаты тестирования показали, что аллокатор `McKusick` обладает более высокой скоростью выделения и освобождения памяти для небольших блоков за счет предварительной организации памяти по классам размера, однако его эффективность снижается при большом количестве запросов разного размера. Аллокатор `FreeList`, напротив, более универсален и хорошо справляется с выделением блоков любых размеров, но требует больше времени на поиск подходящего блока, особенно при высокой фрагментации. Обе реализации обладают высокой простотой использования благодаря единому интерфейсу, который был обеспечен через динамическую загрузку библиотек. Полученные результаты и выводы демонстрируют эффективность каждого из подходов в определенных сценариях и могут быть полезны для дальнейшего применения в реальных проектах.

Код программы

freelist_allocator.c

```
#include <stddef.h>
#include <stdlib.h>
#include <unistd.h>

typedef struct Block {
    size_t size;
    struct Block* next;
} Block;

typedef struct Allocator {
    struct Block* freeList;
    void* memoryStart;
    size_t totalSize;
} Allocator;

Allocator* allocator_create(void *const memory, const size_t
size) {
```

```

if (!memory || size < sizeof(struct Block)) return NULL;

Allocator* allocator = (Allocator*) memory;
allocator->memoryStart = memory;
allocator->totalSize = size;

allocator->freeList = (struct Block*)((char*)memory +
sizeof(Allocator));
allocator->freeList->size = size - sizeof(Allocator);
allocator->freeList->next = NULL;

const char* msg = "FreeList Allocator created
successfully\n";
write(1, msg, 40);

return allocator;
}

void* allocator_alloc(Allocator *const allocator, const size_t
size) {
    if (!allocator || size == 0) {
        const char* msg = "Memory allocation error: invalid
allocator or size.\n";
        write(1, msg, 55);
        return NULL;
    }

    struct Block** current = &allocator->freeList;
    while (*current) {
        if ((*current)->size >= size) {
            if ((*current)->size > size + sizeof(struct Block))
{
                struct Block* remaining = (struct
Block*)((char*)(*current) + sizeof(struct Block) + size);
                remaining->size = (*current)->size - size -
sizeof(struct Block);
                remaining->next = (*current)->next;

```

```

        (*current)->size = size;
        *current = remaining;
    } else {
        *current = (*current)->next;
    }

    const char* allocMsg = "Memory allocated
successfully\n";
    write(1, allocMsg, 30);

    return (void*)((char*)(*current) + sizeof(struct
Block));
}
current = &(*current)->next;
}

const char* errorMsg = "No suitable block found\nMemory
allocation error.\n";
write(1, errorMsg, 46);
return NULL;
}

void allocator_free(Allocator *const allocator, void *const
memory) {
    if (!allocator || !memory) return;

    struct Block* block = (struct Block*)((char*)memory -
sizeof(struct Block));
    block->next = allocator->freeList;
    allocator->freeList = block;

    const char* msg = "Memory freed successfully\n";
    write(1, msg, 27);
}

void allocator_destroy(Allocator* allocator) {
    if (!allocator) {

```

```

        const char* msg = "FreeList Allocator destroy: NULL
pointer\n";
        write(1, msg, 39);
        return;
    }
    const char* msg = "FreeList Allocator destroyed
successfully\n";
    write(1, msg, 42);
}

```

mckusick allocator.c

```

#include <stddef.h>
#include <stdint.h>
#include <unistd.h>

#define NUM_CLASSES 8

typedef struct Block {
    struct Block* next;
    size_t size;
} Block;

typedef struct Allocator {
    struct Block* freeLists[NUM_CLASSES];
    void* memoryStart;
    size_t totalSize;
} Allocator;

static size_t get_class(size_t size) {
    size_t classSize = 8;
    for (size_t i = 0; i < NUM_CLASSES; ++i) {
        if (size <= classSize) return i;
        classSize *= 2;
    }
    return NUM_CLASSES - 1;
}

Allocator* allocator_create(void *const memory, const size_t
size) {

```

```

    if (!memory || size < NUM_CLASSES * sizeof(Block)) {
        const char* msg = "McKusick Allocator creation failed:
invalid memory or size\n";
        write(1, msg, 59);
        return NULL;
    }

    Allocator* allocator = (Allocator*) memory;
    allocator->memoryStart = memory;
    allocator->totalSize = size;

    for (size_t i = 0; i < NUM_CLASSES; ++i) {
        allocator->freeLists[i] = NULL;
    }

    void* currentMemory = (void*)(allocator + 1);
    for (size_t i = 0; i < NUM_CLASSES; ++i) {
        size_t blockSize = 8 * (1 << i);
        while ((size_t)(currentMemory) + blockSize <=
(size_t)(allocator->memoryStart) + allocator->totalSize) {
            Block* block = (Block*)currentMemory;
            block->size = blockSize;
            block->next = allocator->freeLists[i];
            allocator->freeLists[i] = block;

            currentMemory = (void*)((size_t)currentMemory +
blockSize);
        }
    }

    const char* msg = "McKusick Allocator created
successfully\n";
    write(1, msg, 40);

    return allocator;
}

```

```

void* allocator_alloc(Allocator *const allocator, const size_t
size) {
    if (!allocator || size == 0) {
        const char* msg = "Allocation failed: invalid allocator
or size\n";
        write(1, msg, 45);
        return NULL;
    }

    size_t classIndex = get_class(size);
    if (classIndex >= NUM_CLASSES) {
        const char* msg = "Allocation failed: size exceeds
maximum class\n";
        write(1, msg, 47);
        return NULL;
    }

    Block* block = allocator->freeLists[classIndex];
    if (block) {
        allocator->freeLists[classIndex] = block->next;

        const char* msg = "Memory allocated successfully\n";
        write(1, msg, 30);

        return (void*)block;
    }

    size_t classSize = 8 * (1 << classIndex);

    Block* newBlock = (Block*)allocator->memoryStart;
    allocator->memoryStart = (void*)((size_t)allocator->
memoryStart + classSize);

    newBlock->size = classSize;
    newBlock->next = NULL;

    const char* msg = "New block allocated\n";
    write(1, msg, 20);

```

```

    return (void*)newBlock;
}

void allocator_free(Allocator *const allocator, void *const
memory) {
    if (!allocator || !memory) {
        const char* msg = "Free failed: invalid allocator or
memory\n";
        write(1, msg, 41);
        return;
    }

    Block* block = (Block*)memory;
    size_t classIndex = get_class(block->size);
    block->next = allocator->freeLists[classIndex];
    allocator->freeLists[classIndex] = block;

    const char* msg = "Memory freed successfully\n";
    write(1, msg, 27);
}

void allocator_destroy(Allocator* allocator) {
    if (!allocator) {
        const char* msg = "McKusick Allocator destroy: NULL
pointer\n";
        write(1, msg, 40);
        return;
    }

    const char* msg = "McKusick Allocator destroyed
successfully\n";
    write(1, msg, 43);
}

```

main.c

```

#include <stddef.h>
#include <dlfcn.h>

```

```

#include <unistd.h>
#include <sys/mman.h>
#include <stdlib.h>

void ptr_to_str(void* ptr, char* buf) {
    unsigned long addr = (unsigned long)ptr;
    char hex_digits[] = "0123456789abcdef";
    buf[0] = '0';
    buf[1] = 'x';

    for (int i = 15; i >= 2; i--) {
        buf[i] = hex_digits[addr & 0xF];
        addr >>= 4;
    }

    buf[16] = '\0';
}

typedef struct Allocator {
    struct Block* freeList;
    void* memoryStart;
    size_t totalSize;
} Allocator;

typedef Allocator* (*allocator_create_func)(void *const memory,
const size_t size);
typedef void (*allocator_destroy_func)(Allocator *const
allocator);
typedef void* (*allocator_alloc_func)(Allocator *const
allocator, const size_t size);
typedef void (*allocator_free_func)(Allocator *const allocator,
void *const memory);

allocator_create_func allocator_create = NULL;
allocator_destroy_func allocator_destroy = NULL;
allocator_alloc_func allocator_alloc = NULL;
allocator_free_func allocator_free = NULL;

```

```

Allocator* fallback_allocator_create(void *const memory, const
size_t size) {
    (void)memory;
    (void)size;
    Allocator* dummy = mmap(NULL, sizeof(Allocator), PROT_READ |
PROT_WRITE, MAP_ANON | MAP_PRIVATE, -1, 0);
    return dummy;
}

void fallback_allocator_destroy(Allocator *const allocator) {
    if (allocator) munmap(allocator, sizeof(Allocator));
}

void* fallback_allocator_alloc(Allocator *const allocator, const
size_t size) {
    (void)allocator;
    return mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_ANON |
MAP_PRIVATE, -1, 0);
}

void fallback_allocator_free(Allocator *const allocator, void
*const memory) {
    (void)allocator;
    if (memory) munmap(memory, 0);
}

void load_allocator_library(const char* library_path) {
    void* handle = dlopen(library_path, RTLD_LAZY);
    if (!handle) {
        const char* errorMsg = "Error loading library. Falling
back to system allocator.\n";
        write(1, errorMsg, 55);
        allocator_create = fallback_allocator_create;
        allocator_destroy = fallback_allocator_destroy;
        allocator_alloc = fallback_allocator_alloc;
        allocator_free = fallback_allocator_free;
        return;
    }
}

```

```

    allocator_create = (allocator_create_func)dlsym(handle,
"allocator_create");
    allocator_destroy = (allocator_destroy_func)dlsym(handle,
"allocator_destroy");
    allocator_alloc = (allocator_alloc_func)dlsym(handle,
"allocator_alloc");
    allocator_free = (allocator_free_func)dlsym(handle,
"allocator_free");

    if (!allocator_create || !allocator_destroy ||
!allocator_alloc || !allocator_free) {
        const char* symbolError = "Error loading symbols.
Falling back to system allocator.\n";
        write(1, symbolError, 58);
        dlclose(handle);
        allocator_create = fallback_allocator_create;
        allocator_destroy = fallback_allocator_destroy;
        allocator_alloc = fallback_allocator_alloc;
        allocator_free = fallback_allocator_free;
        return;
    }

    const char* successMsg = "Library loaded successfully.\n";
    write(1, successMsg, 29);
}

int main(int argc, char* argv[]) {
    const size_t memory_size = 4096;

    void* memory = mmap(NULL, memory_size, PROT_READ |
PROT_WRITE, MAP_ANON | MAP_PRIVATE, -1, 0);
    if (memory == MAP_FAILED) {
        const char* mmapError = "Failed to allocate memory using
mmap.\n";
        write(1, mmapError, 39);
        return 1;
    }
}

```

```

    if (argc < 2) {
        const char* noLibraryMsg = "No library path provided.
Using system allocator.\n";
        write(1, noLibraryMsg, 52);
        allocator_create = fallback_allocator_create;
        allocator_destroy = fallback_allocator_destroy;
        allocator_alloc = fallback_allocator_alloc;
        allocator_free = fallback_allocator_free;
    } else {
        load_allocator_library(argv[1]);
    }

    Allocator* allocator = allocator_create(memory,
memory_size);
    if (!allocator) {
        const char* createError = "Failed to create
allocator.\n";
        write(1, createError, 28);
        return 1;
    }

    void* ptr1 = allocator_alloc(allocator, 64);
    if (ptr1) {
        const char* allocMsg1 = "64 bytes allocated
successfully. Pointer address: ";
        write(1, allocMsg1, 50);
        char ptr1Str[18];
        ptr_to_str(ptr1, ptr1Str);
        write(1, ptr1Str, 18);
        const char* endMsg1 = "\n";
        write(1, endMsg1, 1);
        allocator_free(allocator, ptr1);
    } else {
        const char* allocError1 = "Failed to allocate 64
bytes.\n";
        write(1, allocError1, 30);
    }
}

```

```

void* ptr2 = allocator_alloc(allocator, 128);
if (ptr2) {
    const char* allocMsg2 = "128 bytes allocated
successfully. Pointer address: ";
    write(1, allocMsg2, 51);
    char ptr2Str[18];
    ptr_to_str(ptr2, ptr2Str);
    write(1, ptr2Str, 18);
    const char* endMsg2 = "\n";
    write(1, endMsg2, 1);
    allocator_free(allocator, ptr2);
} else {
    const char* allocError2 = "Failed to allocate 128
bytes.\n";
    write(1, allocError2, 31);
}

allocator_destroy(allocator);
return 0;
}

```

Протокол работы программы

Тестирование:

```

-migermanenko@MacBook-Pro-Matvej lab4 % make
gcc -Wall -Werror -g -fPIC -c main.c -o main.o
gcc main.o -o main -ldl
-migermanenko@MacBook-Pro-Matvej lab4 % ./main ./mckusick_allocator.so
Library loaded successfully.
McKusick Allocator created successfully
New block allocated
64 bytes allocated successfully. Pointer address: 0x00000100cb0000
Memory freed successfully
New block allocated
128 bytes allocated successfully. Pointer address: 0x00000100cb0040
Memory freed successfully
McKusick Allocator destroyed successfully
-migermanenko@MacBook-Pro-Matvej lab4 % ./main ./freelist_allocator.so
Library loaded successfully.
FreeList Allocator created successfully
Memory allocated successfully
64 bytes allocated successfully. Pointer address: 0x00000104860078
Memory freed successfully
Memory allocated successfully

```

```

128 bytes allocated successfully. Pointer address: 0x00000104860108
Memory freed successfully
FreeList Allocator destroyed successfully
-migermanenko@MacBook-Pro-Matvej lab4 % make clean
rm -f main.o freelist_allocator.o mckusick_allocator.o main freelist_allocator.so
mckusick_allocator.so

```

Dtruss:

```

migermanenko@MacBook-Pro-Matvej lab4 % sudo dtruss -f ./main ./freelist_allocator.so

```

```

PID/THRD SYSCALL(args) = return
Library loaded successfully.
FreeList Allocator created successfully
Memory allocated successfully
64 bytes allocated successfully. Pointer address: 0x0000010429c078
Memory freed successfully
Memory allocated successfully
128 bytes allocated successfully. Pointer address: 0x0000010429c108
Memory freed successfully
FreeList Allocator destroyed successfully
1295/0x2db8: fork() = 0 0
1295/0x2db8: munmap(0x1043EC000, 0x84000) = 0 0
1295/0x2db8: munmap(0x104470000, 0x8000) = 0 0
1295/0x2db8: munmap(0x104478000, 0x4000) = 0 0
1295/0x2db8: munmap(0x10447C000, 0x4000) = 0 0
1295/0x2db8: munmap(0x104480000, 0x48000) = 0 0
1295/0x2db8: munmap(0x1044C8000, 0x4C000) = 0 0
1295/0x2db8: crossarch_trap(0x0, 0x0, 0x0) = -1 Err#45
1295/0x2db8: open("./0", 0x100000, 0x0) = 3 0
1295/0x2db8: fcntl(0x3, 0x32, 0x16BBB7098) = 0 0
1295/0x2db8: close(0x3) = 0 0
1295/0x2db8: fsgetpath(0x16BBB70A8, 0x400, 0x16BBB7088) = 45 0
1295/0x2db8: fsgetpath(0x16BBB70B8, 0x400, 0x16BBB7098) = 14 0
1295/0x2db8: csrctl(0x0, 0x16BBB74BC, 0x4) = -1 Err#1
1295/0x2db8: __mac_syscall(0x187B7BC12, 0x2, 0x16BBB7400) = 0 0
1295/0x2db8: csrctl(0x0, 0x16BBB74AC, 0x4) = -1 Err#1
1295/0x2db8: __mac_syscall(0x187B78A45, 0x5A, 0x16BBB7440) = 0 0
1295/0x2db8: sysctl([unknown, 3, 0, 0, 0, 0] (2), 0x16BBB69A8, 0x16BBB69A0, 0x187B7A738,
0xD) = 0 0
1295/0x2db8: sysctl([CTL_KERN, 157, 0, 0, 0, 0] (2), 0x16BBB6A58, 0x16BBB6A50, 0x0, 0x0)
= 0 0
1295/0x2db8: open("/0", 0x20100000, 0x0) = 3 0
1295/0x2db8: openat(0x3, "System/Cryptexes/OS/0", 0x100000, 0x0) = 4 0
1295/0x2db8: dup(0x4, 0x0, 0x0) = 5 0
1295/0x2db8: fstatat64(0x4, 0x16BBB6531, 0x16BBB64A0) = 0 0
1295/0x2db8: openat(0x4, "System/Library/dyld/0", 0x100000, 0x0) = 6 0
1295/0x2db8: fcntl(0x6, 0x32, 0x16BBB6530) = 0 0
1295/0x2db8: dup(0x6, 0x0, 0x0) = 7 0
1295/0x2db8: dup(0x5, 0x0, 0x0) = 8 0
1295/0x2db8: close(0x3) = 0 0
1295/0x2db8: close(0x5) = 0 0
1295/0x2db8: close(0x4) = 0 0
1295/0x2db8: close(0x6) = 0 0
1295/0x2db8: __mac_syscall(0x187B7BC12, 0x2, 0x16BBB6F20) = 0 0
1295/0x2db8: shared_region_check_np(0x16BBB6B40, 0x0, 0x0) = 0 0
1295/0x2db8: fsgetpath(0x16BBB70C0, 0x400, 0x16BBB6FE8) = 82 0

```

```

1295/0x2db8: fcntl(0x8, 0x32, 0x16BBB70C0)          = 0 0
1295/0x2db8: close(0x8)                             = 0 0
1295/0x2db8: close(0x7)                             = 0 0
1295/0x2db8: getfsstat64(0x0, 0x0, 0x2)             = 11 0
1295/0x2db8: getfsstat64(0x104248050, 0x5D28, 0x2)   = 11 0
1295/0x2db8: getattrlist("/^0", 0x16BBB7000, 0x16BBB6F70) = 0 0
1295/0x2db8:
stat64("/System/Volumes/Preboot/Cryptexes/OS/System/Library/dyld/dyld_shared_cache_arm64e\0", 0x16BBB7360, 0x0) = 0 0
dtrace: error on enabled probe ID 1696 (ID 845: syscall::stat64:return): invalid address (0x0) in
action #12 at DIF offset 12
1295/0x2db8: stat64("/Users/migermanenko/Documents/C/OC/Lab4/main\0", 0x16BBB6810, 0x0)
= 0 0
1295/0x2db8: open("/Users/migermanenko/Documents/C/OC/Lab4/main\0", 0x0, 0x0)      = 3 0
1295/0x2db8: mmap(0x0, 0x88D8, 0x1, 0x40002, 0x3, 0x0)      = 0x104248000 0
1295/0x2db8: fcntl(0x3, 0x32, 0x16BBB6928)          = 0 0
1295/0x2db8: close(0x3)                                = 0 0
1295/0x2db8: munmap(0x104248000, 0x88D8)              = 0 0
1295/0x2db8: stat64("/Users/migermanenko/Documents/C/OC/Lab4/main\0", 0x16BBB6D80, 0x0)
= 0 0
1295/0x2db8: stat64("/usr/lib/libSystem.B.dylib\0", 0x16BBB5CD0, 0x0)      = -1 Err#2
1295/0x2db8: stat64("/System/Volumes/Preboot/Cryptexes/OS/usr/lib/libSystem.B.dylib\0",
0x16BBB5C80, 0x0)      = -1 Err#2
1295/0x2db8: open("/Users/migermanenko/Documents/C/OC/Lab4/main\0", 0x0, 0x0)      = 3 0
1295/0x2db8: __mac_syscall(0x187B7BC12, 0x2, 0x16BBB43D0)      = 0 0
1295/0x2db8: map_with_linking_np(0x16BBB42B0, 0x1, 0x16BBB42E0)      = 0 0
1295/0x2db8: close(0x3)                                = 0 0
1295/0x2db8: mprotect(0x10423C000, 0x4000, 0x1)          = 0 0
1295/0x2db8: open("/dev/dtracehelper\0", 0x2, 0x0)        = 3 0
1295/0x2db8: ioctl(0x3, 0x80086804, 0x16BBB3988)          = 0 0
1295/0x2db8: close(0x3)                                = 0 0
1295/0x2db8: shared_region_check_np(0xFFFFFFFFFFFFFFFF, 0x0, 0x0)      = 0 0
1295/0x2db8: access("/AppleInternal/XBS/.isChrooted\0", 0x0, 0x0)      = -1 Err#2
1295/0x2db8: bsdtthread_register(0x187E7E0F4, 0x187E7E0E8, 0x4000)      = 1073746399 0
1295/0x2db8: getpid(0x0, 0x0, 0x0)                      = 1295 0
1295/0x2db8: shm_open(0x187D15F41, 0x0, 0xFFFFFFFF87EBC000)      = 3 0
1295/0x2db8: fstat64(0x3, 0x16BBB4000, 0x0)              = 0 0
1295/0x2db8: mmap(0x0, 0x8000, 0x1, 0x40001, 0x3, 0x0)      = 0x104250000 0
1295/0x2db8: close(0x3)                                = 0 0
1295/0x2db8: csops(0x50F, 0x0, 0x16BBB413C)              = 0 0
1295/0x2db8: ioctl(0x2, 0x4004667A, 0x16BBB40AC)          = 0 0
1295/0x2db8: mprotect(0x104260000, 0x4000, 0x0)           = 0 0
1295/0x2db8: mprotect(0x10426C000, 0x4000, 0x0)           = 0 0
1295/0x2db8: mprotect(0x104270000, 0x4000, 0x0)           = 0 0
1295/0x2db8: mprotect(0x10427C000, 0x4000, 0x0)           = 0 0
1295/0x2db8: mprotect(0x104280000, 0x4000, 0x0)           = 0 0
1295/0x2db8: mprotect(0x10428C000, 0x4000, 0x0)           = 0 0
1295/0x2db8: mprotect(0x104258000, 0xC8, 0x1)            = 0 0
1295/0x2db8: mprotect(0x104258000, 0xC8, 0x3)            = 0 0
1295/0x2db8: mprotect(0x104258000, 0xC8, 0x1)            = 0 0
1295/0x2db8: mprotect(0x104290000, 0x4000, 0x1)          = 0 0
1295/0x2db8: mprotect(0x104294000, 0xC8, 0x1)            = 0 0
1295/0x2db8: mprotect(0x104294000, 0xC8, 0x3)            = 0 0
1295/0x2db8: mprotect(0x104294000, 0xC8, 0x1)            = 0 0
1295/0x2db8: mprotect(0x104258000, 0xC8, 0x3)            = 0 0

```

```

1295/0x2db8: mprotect(0x104258000, 0xC8, 0x1)      = 0 0
1295/0x2db8: mprotect(0x104290000, 0x4000, 0x3)    = 0 0
1295/0x2db8: mprotect(0x104290000, 0x4000, 0x1)    = 0 0
1295/0x2db8: issetugid(0x0, 0x0, 0x0)            = 0 0
1295/0x2db8: getentropy(0x16BBB3708, 0x20, 0x0)   = 0 0
1295/0x2db8: getatrlist("/Users/migermanenko/Documents/C/OC/Lab4/main\0", 0x16BBB3FA0,
0x16BBB3FBC)    = 0 0
1295/0x2db8: access("/Users/migermanenko/Documents/C/OC/Lab4\0", 0x4, 0x0)      = 0 0
1295/0x2db8: open("/Users/migermanenko/Documents/C/OC/Lab4\0", 0x0, 0x0)        = 3 0
1295/0x2db8: fstat64(0x3, 0x130E04500, 0x0)      = 0 0
1295/0x2db8: csrctl(0x0, 0x16BBB418C, 0x4)        = 0 0
1295/0x2db8: fcntl(0x3, 0x32, 0x16BBB3E88)        = 0 0
1295/0x2db8: close(0x3)                          = 0 0
1295/0x2db8: open("/Users/migermanenko/Documents/C/OC/Lab4/Info.plist\0", 0x0, 0x0) =
-1 Err#2
1295/0x2db8: proc_info(0x2, 0x50F, 0xD)           = 64 0
1295/0x2db8: csops_audittoken(0x50F, 0x10, 0x16BBB4210) = 0 0
1295/0x2db8: sysctl([unknown, 3, 0, 0, 0, 0] (2), 0x16BBB4568, 0x16BBB4560, 0x18B5A8D3A,
0x15)          = 0 0
1295/0x2db8: sysctl([CTL_KERN, 155, 0, 0, 0, 0] (2), 0x16BBB45F8, 0x16BBB45F0, 0x0, 0x0)
= 0 0
1295/0x2db8: mmap(0x0, 0x1000, 0x3, 0x41002, 0xFFFFFFFFFFFFFFFF, 0x0)      =
0x10429C000 0
1295/0x2db8: open("./freelist_allocator.so\0", 0x0, 0x0)      = 3 0
1295/0x2db8: fcntl(0x3, 0x32, 0x16BBC60C8)                    = 0 0
1295/0x2db8: close(0x3)                                        = 0 0
1295/0x2db8: stat64("./freelist_allocator.so\0", 0x16BBC5C30, 0x0) = 0 0
1295/0x2db8: stat64("./freelist_allocator.so\0", 0x16BBC5660, 0x0) = 0 0
1295/0x2db8: open("./freelist_allocator.so\0", 0x0, 0x0)      = 3 0
1295/0x2db8: mmap(0x0, 0x84D8, 0x1, 0x40002, 0x3, 0x0)        = 0x1042A0000 0
1295/0x2db8: fcntl(0x3, 0x32, 0x16BBC5778)                    = 0 0
1295/0x2db8: close(0x3)                                        = 0 0
1295/0x2db8: open("/Users/migermanenko/Documents/C/OC/Lab4/freelist_allocator.so\0", 0x0,
0x0)          = 3 0
1295/0x2db8: fstat64(0x3, 0x16BBC4E20, 0x0)                    = 0 0
1295/0x2db8: fcntl(0x3, 0x61, 0x16BBC5418)                     = 0 0
1295/0x2db8: fcntl(0x3, 0x62, 0x16BBC5418)                     = 0 0
1295/0x2db8: mmap(0x1042AC000, 0x4000, 0x5, 0x40012, 0x3, 0x0) = 0x1042AC000 0
1295/0x2db8: mmap(0x1042B0000, 0x4000, 0x3, 0x40012, 0x3, 0x4000) = 0x1042B0000
0
1295/0x2db8: mmap(0x1042B4000, 0x4000, 0x1, 0x40012, 0x3, 0x8000) = 0x1042B4000
0
1295/0x2db8: close(0x3)                                        = 0 0
1295/0x2db8: munmap(0x1042A0000, 0x84D8)                      = 0 0
1295/0x2db8: open("/Users/migermanenko/Documents/C/OC/Lab4/freelist_allocator.so\0", 0x0,
0x0)          = 3 0
1295/0x2db8: close(0x3)                                        = 0 0
1295/0x2db8: mprotect(0x1042B0000, 0x4000, 0x1)              = 0 0
1295/0x2db8: write(0x1, "Library loaded successfully.\n\0", 0x1D) = 29 0
1295/0x2db8: write(0x1, "FreeList Allocator created successfully\n\0", 0x28) = 40 0
1295/0x2db8: write(0x1, "Memory allocated successfully\n\0", 0x1E) = 30 0
1295/0x2db8: write(0x1, "64 bytes allocated successfully. Pointer address: \0", 0x32) = 50 0
1295/0x2db8: write(0x1, "0x0000010429c078\0", 0x12)          = 18 0
1295/0x2db8: write(0x1, "\n\0", 0x1)                        = 1 0
1295/0x2db8: write(0x1, "Memory freed successfully\n\0", 0x1B) = 27 0

```

```

1295/0x2db8: write(0x1, "Memory allocated successfully\n\0", 0x1E)          = 30 0
1295/0x2db8: write(0x1, "128 bytes allocated successfully. Pointer address: \0", 0x33)    =
51 0
1295/0x2db8: write(0x1, "0x0000010429c108\0", 0x12)          = 18 0
1295/0x2db8: write(0x1, "\n\0", 0x1)          = 1 0
1295/0x2db8: write(0x1, "Memory freed successfully\n\0", 0x1B)          = 27 0
1295/0x2db8: write(0x1, "FreeList Allocator destroyed successfully\n\0", 0x2A)          = 42 0

```

Вывод

В данной работе была реализована программа, демонстрирующая использование пользовательских аллокаторов памяти, подключаемых через динамические библиотеки с использованием `dlopen` и `dlsym`. Программа позволяет создавать, выделять, освобождать и уничтожать аллокаторы, а также выводить адреса выделенных блоков памяти для отладки. Системные вызовы `mmap`, `munmap` и `write` обеспечивают управление памятью и вывод информации без использования стандартных библиотек ввода-вывода. Реализация демонстрирует гибкость и эффективность работы с памятью на низком уровне, что может быть полезно для задач оптимизации и изучения системных механизмов.